P2P File Sharing Application
System Functional Specification

CMP2204 Term Project Spring 2026

1 System Definition

P2P File Sharing application consists of two major phases: *(i)* discovering all available users in the network and the contents they carry, and *(ii)* downloading a content (all chunks of this content) from the users in the network. Figure 1 demonstrates the four processes that together make up the P2P file sharing application; the left two processes correspond to the discovery of content and peers; the right two processes correspond to transfering content between pairs.

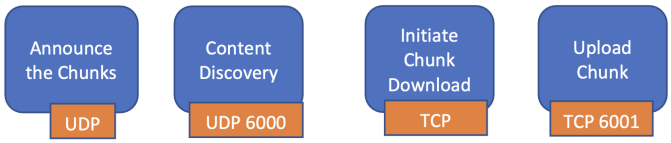

Figure 1: The four processes of the peer-to-peer file exchange application

1.1 Operational Scenarios

The following are the use cases supported by the P2P File Sharing Application:

Service Announcement - Chunk Discovery: Upon connecting to the Local Area Network, every peer starts to *periodically, every 8 seconds* broadcast their presence. They insert their name and the list of all files they have in the message in JSON (JavaScript Object Notation) format. (Figure 2).

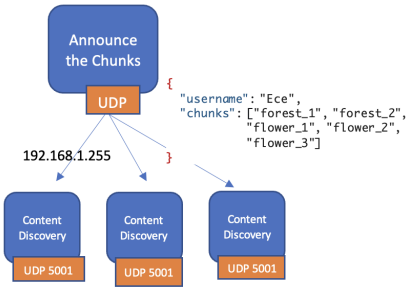

Figure 2: The Chunk Announcer broadcasts its chunks to all nodes in the network.

Upon connecting to the Local Area Network, every node also starts listening for peer announcements in the LAN. Upon hearing an announcement, the payload is parsed; the username and the contents that this user has are stored in a local dictionary.

Viewing available contents: The application end user can view the list of available users in the network and the contents they carry. The list only contains the users and contents that were discovered in the last 2 minutes (120 seconds). (This requires updating the table each time a new announcement has been received.)

Initiating a file download: The end user specifies a content name to download. The Chunk Downloader process (TCP client) looks up its local dictionary to see which users (IP addresses) carry the chunks of that content. Then it (automatically, without user intervention) launches TCP sessions with these IP addresses to download each subpart of that file (Figure 3). When the download of a chunk is complete, the TCP session is closed. A new TCP session will be initiated with another user for another chunk, until all chunks of the file have been downloaded

Figure 3: The Chunk Announcer broadcasts its chunks to all nodes in the network.

Serving (parts of) files: Every user has one original file, which, they will divide into 3 chunks. In addition, every user will altruistically serve all pieces of other users' files that they have downloaded. When a new TCP connection request is received, this connection request is immediately accepted; the TCP server will parse the received message to understand which chunk is requested, then sends the requested file to the requesting node over this TCP connection. For security, the sender will *encrypt* and *encode* the chunk before inserting it into the JSON payload; the receiver must decrypt it. The UI of the sender will display the requesting user's name and the chunk that is being requested.

Security: TCP sender and receiver can generate a shared secret key using the Diffie-Hellman key exchange algorithm. We will study Diffie-Hellman key exchange in Week 11. Diffie-Hellman is a mathematical operation, it is used to generate a *shared key* from two random numbers exchanged by the communicating end nodes. TCP sender (*i.e.,* Chunk_Uploader module) will use that key in encrypting its payloads. In this project, the Diffie-Hellman parameters p and g **shall** be used as: $p = 907$ and $g = 7$.

For encryption, we will use an external library, `pyDes`. You will need to install it in your environment using a command like `pip install pyDes`. Note that the shared key generated upon Diffie-Hellman key exchange is an *integer*. However, block ciphers like DES *do not accept integers as keys*. They operate strictly on *byte arrays*. If you try to directly pass the generated shared key into *pyDes*, the program will crash with a type error.

Furthermore, `pyDes` has a rigid requirement regarding length: DES encryption requires *a key that is exactly* **8 bytes** *long*. If your Diffie-Hellman-generated shared secret is shorter than that (*e.g.* 456), and you simply convert that to a byte string (b'456') that is only 3 bytes, `pyDes` will throw a ValueError: Invalid DES key size. To workaround that problem, you need to pad the beginning of the string with zeros to reach 8 characters, then encode to bytes. However, if the generated shared key is longer than 8 bytes, then you also need to truncate it. It would be safe to always pad and truncate to the first 8 bytes like this:

```
shared_secret_int = ..... # in your project it will be computed by Diffie-Hellman
# Convert to string, pad with zeros to 8 chars, enforce an 8-char limit, then encode to bytes
des_key_string = str(shared_secret_int).zfill(8)[:8]
des_key_bytes = des_key_string.encode('utf-8')
```

Separately, note that once you encrypt your data, the output will be raw bytes. Hence, you need to convert the encrypted text into a string using *Base64 encoding* before inserting it into your JSON payload for transmission, and reverse the process upon receipt. For this, you need to include the base64 encoding library: `import base64`. UTF decoding formally changes the Python variable type from bytes to str (e.g., 'SGVsbG8=') so that json.dumps() will accept it.

The pyDes encryption at Chunk Uploader can look like the following; please note that you will have to reverse these steps at the Chunk Downloader upon reception.

```
# Assume: raw_imgchunk_bytes is the binary containing raw bytes of your image chunk
encrypted_bytes = pyDes.des(des_key_bytes, pyDes.ECB, pad=None,padmode=pyDes.PAD_PKCS5).encrypt(raw_imgchunk_bytes)
# Next, encode to Base64 to safely send inside the JSON payload
encoded_chunk_string = base64.b64encode(encrypted_bytes).decode('utf-8')
```

If the chunk will be sent unencrypted, the JSON will still need to convert the raw image bytes into Base64 string and tag it as text, as shown below.

```
json_safe_string  = base64.b64encode(raw_image_chunk_bytes).decode('utf-8')
```

Download history: End user can view the download and upload history (timestamp, name of peer, content name, sent/received, chunk index).

2 Requirements

Throughout this section, the term **shall** indicates an obligatory requirement that must be met to comply with the specification, whereas the term **may** indicates an item that is truly optional.

2.1 Chunk Announcement Requirements

Req. #	**Requirement**
2.1.0-A	When launched, Chunk_Announcer **shall** ask the user to specify their username, and the file it will initially host (*i.e.,* that user's original file). Chunk_Announcer **shall** divide the specified file into N-byte chunks and store them as separate files with indexed naming. (The code for this will be provided to you.) (Note that these chunks will not have .png suffix.) Once this is done, Chunk_Announcer **shall** display a message on terminal, informing the end user about the number of chunks, and stating it is starting to announce these files. To simplify the design, let's assume each file in our system always has 3 chunks. It shall also store the username locally.
2.1.0-B	After preparing the chunks, Chunk_Announcer **shall** start to periodically send broadcast UDP messages in the network to announce its presence. The period **shall** be once per **8 seconds**. For the broadcast IP address, please use 192.168.1.255.
2.1.0-C	Chunk_Announcer **shall** be able to read the names of the files under a specified directory, and insert them into the message in JSON array format.
2.1.0-D	Chunk_Announcer's periodic broadcasts **shall** contain a JSON including the username specified by the end user and the list of hosted files. It is very important that the keys are exactly "username" (all lowercase, no underscore) and "chunks" (all lowercase) and that the format is a valid JSON format; otherwise you may have parsing issues when working with your peers. An example would look like: {"username": "Ece", "chunks": ["forest_1", "forest_2", "forest_3", "flower_2", "bee_1", "city_1", "city_2"]}.

2.2 Content Discovery Requirements

Req. #	Requirement
2.2.0-A	Content_Discovery **shall** listen for UDP broadcast messages on port **6000**.
2.2.0-B	Upon receiving a broadcast message, Content_Discovery process **shall**: *(i)* parse the message contents using a JSON parser in Python, *(ii)* get the UDP broadcast sender's IP address using recvfrom() method.
2.2.0-C	Content_Discovery **shall** maintain a dictionary to map IP addresses to usernames for a user-friendly UI. The obtained *username* and the IP address retrieved in the UDP recvfrom() call **shall** be stored in a dictionary. The dictionary keys **shall** be the IP address (that you fetched using recvfrom()). This dictionary **shall** be shared with the Chunk_Uploader and Chunk_Downloader processes. For this, you **may** store it in a local text file that is shared between these processes. You need to consult this dictionary whenever you will write on the console "from whom" you are requesting a content. Instead of displaying the IP address, you should display the username.
2.2.0-D	Content_Discovery **shall** also store the list of files (parsed from the JSON message) in a dictionary – a separate dictionary than the one above. Let's call this *the content dictionary*. In this dictionary, the dictionary keys **shall** be the *content chunk name* (notice – chunk, not content itself, *e.g.,* forest_1) and the value **shall** be an array containing the list of users having that chunk. This dictionary **shall** be shared with the Chunk_Downloader process. You **may** store it in a local text file that is shared between the Content_Discovery and Chunk_Downloader components.
2.2.0-E	Additional to the IP Address - username dictionary, you **shall** also maintain a username - IP Address dictionary, to map names to IP addresses. This one will be used for initiating TCP sessions with the respective IP address when downloading a chunk – you lookup which users have that chunk, then lookup the IP address of the user.
2.2.0-F	Upon every insertion into *content dictionary*, Content_Discovery **shall** display the detected username and their hosted content on the console (*e.g.,* "Ece : vid_1, vid_2, vid_3"). This would provide better user experience. This would also help you with debugging.
2.2.0-G	You should wipe off *the content dictionary* every minute, so that only the recently discovered content will be displayed to the end user. I will not ask you to precisely keep track of the timestamps when each content is detected; one minute is a coarsely rough estimate of recency.

2.3 Chunk Downloader Requirements

Req. #	Requirement
2.3.0-A	When launched, Chunkt_Downloader **shall** prompt the user to specify whether they would like to view available chunks, initiate content download, or view download history. The user interactions and GUI (you can assume Terminal/console) that you prepare is up to you. User **shall** be able to specify "View Contents", "Download Content", or "History".
2.3.0-B	When user specifies "View Content", Chunk_Downloader **shall** display the list of content files – not chunks – available in the network. Instead of listing forest_1.png, flower_2.png, etc., the process must list forest.png and flower.png. To display a content name, your code does *NOT* have to ensure all its chunks are available in the network. You can include the content name if at least one of its chunks is found in one of the nodes in the network. Note that to display the content names properly and without replicas you need to write a special code for it.
2.3.0-C	When user specifies "Download Content", Chunk_Downloader **shall** prompt the user to specify which content it wants to download. (For UI purposes it doesn't matter if the user chooses "forest.png" or "forest", that depends on your internal implementation of the UI and underlying dictionary; but the chunks that will be requested shall be in the format "forest_1", "forest_2", "forest_3".) When user specifies a specific content, Chunk_Downloader **shall** first ask the user whether they prefer to download it securely or not, and remember the user's choice. For the user-entered filename, (for example "forest.png"), Chunk_Downloader **shall** initiate 3 sequential download procedures for each chunk of this file, as described in the following requirements.
2.3.0-D	To download each of the 3 chunks of the content that the user wants to download, Chunk_Downloader **shall** lookup its *content dictionary* to fetch the list of IP addresses having a certain chunk; it'll lookup the dictionary with the key set to chunk name (e.g. "forest_1"). Chunk_Downloader **shall** try downloading the chunk from the first IP address in the array that is in the *content dictionary* for this chunk name.
2.3.0-E	For downloading each chunk, Chunk_Downloader **shall** initiate a TCP session with the IP address (determined in the previous requirement). The message **shall** contain the right JSON for requesting secure or unsecure download of that chunk. Please heed the next few requirements regarding the JSON format. Additionally, on the UI, a message **shall** be displayed indicating which chunk is being requested from which user at which timestamp. This will help you with debugging and improve UI.

Req. #	Requirement
2.3.0-F	[**Secure Download**] If the user picked "*secure*", then the sender (*i.e.,* Chunk_Uploader) will need to encrypt the chunk with a shared secret key. As the side initiating the download, Chunk_Downloader **shall** initiate key exchange with the remote device. So when the user prompts "*secure*", Chunk_Downloader **shall** randomly generate a random number (*i.e., key*) and send it to the user in a message containing a JSON including the key. The JSON **shall** look exactly like this: '{"key": "XXX"}' (with XXX replaced by the number). The random number you generate must be smaller than p. The random number should not be 1 or $p-1$ (they generate a mathematically trivial shared secret); you can use python command : *random.randint(2, p-2)*. The end device will in return send a message including a JSON containing their random number, say Y. Upon receiving the end host's random number Y, Chunk_Downloader **shall** compute the *shared_secred_key* using Diffie-Hellman key exchange mechanism. Once the shared key is computed, Chunk_Downloader **shall** store it and use it later for decrypting the chunk received from this user in this TCP session. Chunk_Downloader **shall** then request the chunk name in a JSON like this: {"requested_secured_content": "forest_1" }. It is very important that the JSON keys exactly match the specifications and the format is a valid JSON format; otherwise you may have parsing issues when working with your peers.
2.3.0-G	[**Unsecure Download**] If the user picked "*unsecure*", then Chunk_Downloader can bypass the key generation part and directly generate this JSON to request content: {"requested_content": "forest_1" }.
2.3.0-H	If download of first chunk is successful, Chunk_Downloader **shall** move on to the next chunk (and repeat until all 3 chunks are downloaded). If it is not successful, Chunk_Downloader **shall** inform the end user via the console (*e.g.,* "Chunk [chunk name] cannot be downloaded from [user name]".) Then it **shall** try downloading from the other IP addresses in the array until download of that chunk is successful. If all users in array have been tried and that chunk cannot be downloaded, Chunk_Downloader **shall** display a warning message to the user informing about the problem. (*e.g.,* "CHUNK forest_3 CANNOT BE DOWNLOADED FROM ONLINE PEERS.")
2.3.0-I	After the 3^{rd} chunk is downloaded, Chunk_Downloader **shall** combine these 3 chunks into a single file. (I'll provide the code for this, which you'll integrate in your Chunk_Downloader code.) Once the file is ready, the Chunk_Downloader **shall** inform the user via the terminal that the file has been successfully downloaded. **Please do not remove individual chunks after obtaining the merged content – we may use the chunks on the demo day.**
2.3.0-J	Chunk_Downloader **shall** close a TCP session upon receiving the chunk it requested.
2.3.0-K	Upon receiving each chunk, Chunk_Downloader **shall** log this event: Each entry **shall** specify timestamp, chunk name, received from, marked as "RECEIVED".
2.3.0-L	Chunk_Downloader **shall** dump all downloaded filenames in a Download log (a text file) under the same directory. Each entry **shall** specify timestamp, chunk_name, downloaded_from_IP_address.
2.3.0-M	After a TCP session is closed, Chunk_Downloader **shall** persist; the service **shall not** terminate.

2.4 Chunk Uploader Requirements

Req. #	Requirement
2.4.0-A	Chunk_Uploader **shall** listen for TCP connections on port 6001.
2.4.0-B	Chunk_Uploader **shall** accept TCP connection request before it times out.
2.4.0-C	Chunk_Uploader **shall** parse the JSON in the message to learn whether a *key* is being exchanged or a *content* (*encrypted*/*unencrypted*) is being requested. For this, it **shall** parse the JSON message. *(i)* If the parsed message has a JSON field of "key", then the Chunk_Uploader **shall** receive the remote key, generate its own random number and send it along the same TCP connection in a JSON that looks like: {"key": "XXX"}. Then, the Chunk_Uploader **shall** compute the shared key. *(ii)* If the parsed message has a JSON key of "requested_secured_content', Chunk_Uploader **shall encrypt** and encode it as described in Sec. 1.1. Once the string is ready, Chunk_Downloader **shall** insert it in a JSON message that looks like this: '{ "chunk_name": "forest_1", "encrypted_chunk": "YYY"}' (with YYY replaced by encrypted+encoded text). *(iii)* If the parsed message has a JSON key of "requested_content", Chunk_Uploader **shall** send this file to the requester over the TCP connection, in a JSON message that looks like this: '{ "chunk_name": "forest_1", "data": "YYY"}' (with YYY replaced by the JSON safe string as shown in Sec. 1.1.
2.4.0-D	Upon sending a chunk, Chunk_Uploader **shall** log the file's info in a text file under the same directory. Each entry **shall** specify timestamp, chunk name, recipient's name, marked as "SENT".
2.4.0-E	After a TCP session is closed, Chunk_Uploader **shall** persist; the service **shall not** terminate and **shall** continue to listen on port 6001.